Punto 1

Usando un programa especializado, por ejemplo Mathematica, etc, calcule π con cien mil cifras decimales.

Con la parte decimal forme sucesiones de a 1, de a 2, de a 5, de a 10 cifras.

Verifique si estas sucesiones se pueden asumir como uniformes (usted escoja las pruebas estadísticas que desee pero explíquelas), y si se puede asumir aleatoriedad (usted escoja las pruebas estadísticas que desee pero explíquelas).

Solucion

Cálculo de π con 100,000 Decimales y Análisis de Sucesiones Para este estudio, el objetivo principal es calcular el valor de π con una precisión de 100,000 cifras decimales y, posteriormente, examinar las propiedades de uniformidad y aleatoriedad de las sucesiones formadas por estos dígitos. Se empleará el lenguaje de programación R, utilizando el paquete Rmpfr para manejar la aritmética de precisión arbitraria.

Determinación de la Precisión Necesaria El primer paso es establecer la precisión requerida en bits para asegurar 100,000 dígitos decimales exactos. Esto se logra multiplicando el número deseado de decimales por el logaritmo en base 2 de 10. Se añade un pequeño margen para evitar errores de redondeo.

```r
library(Rmpfr)

# Definir la cantidad de cifras decimales
num_decimales <- 100000

# Calcular los bits necesarios con un margen de seguridad
bits_precision <- ceiling(num_decimales * log2(10)) + 10
print(bits_precision)
```

```
## [1] 332203
```

Como resultado, se necesitarán 332,203 bits para el cálculo.

Generación de π y Extracción de Decimales Una vez definida la precisión, se procede a calcular π utilizando la función Const del paquete Rmpfr. Luego, el valor resultante se convierte a una cadena de caracteres para facilitar la extracción de los dígitos decimales.

```r
# Calcular pi con la precisión especificada
pi_val <- Const("pi", prec = bits_precision)

# Convertir pi a una cadena de caracteres para extraer los decimales
pi_str <- formatMpfr(pi_val, digits = num_decimales + 5)

# Extraer la parte decimal (saltando el "3." inicial)
decimal_pi <- substr(pi_str, 3, num_decimales + 2)
print(decimal_pi)
```

```
## [1] "14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211
```

Para asegurar que se generaron suficientes decimales, se realiza una verificación de la longitud de la cadena obtenida.

```r
if (nchar(decimal_pi) < num_decimales) {
  stop(paste("Insuficientes decimales generados. Se obtuvieron", nchar(decimal_pi),
             "cuando se requerían", num_decimales))
}
```

Con esto, se tiene la parte decimal de π lista para el análisis.

Formación de Sucesiones Numéricas

A partir de los 100,000 decimales de π, se construirán cuatro sucesiones distintas, agrupando los dígitos de a 1, 2, 5 y 10 cifras.

Sucesión de a 1 Cifra

Esta sucesión simplemente toma cada dígito individualmente y los convierte en un vector de enteros.

```r
sucesion_1_cifra <- as.integer(strsplit(decimal_pi, "")[[1]])

cat("Longitud de la sucesión de 1 cifra:", length(sucesion_1_cifra), "\n")
```

```
## Longitud de la sucesión de 1 cifra: 100000
```

```r
cat("Primeros 20 elementos de la sucesión de 1 cifra:\n")
```

```
## Primeros 20 elementos de la sucesión de 1 cifra:
```

```r
print(head(sucesion_1_cifra, 20))
```

```
##  [1] 1 4 1 5 9 2 6 5 3 5 8 9 7 9 3 2 3 8 4 6
```

Sucesiones de 2, 5 y 10 Cifras

Para las sucesiones de múltiples cifras, se utilizan funciones para extraer subcadenas y convertirlas a valores numéricos. Se calcula la cantidad de grupos posibles y se itera para formar cada sucesión.

```r
# Función auxiliar para crear sucesiones
crear_sucesion <- function(cadena, tam_grupo) {
  num_grupos <- floor(nchar(cadena) / tam_grupo)
  indices <- seq(from = 1, by = tam_grupo, length.out = num_grupos)

  sucesion_str <- substring(cadena, indices, indices + tam_grupo - 1)

  # Usar as.numeric para grupos grandes para evitar desbordamientos de as.integer
  if (tam_grupo <= 9) { # Para grupos de hasta 9 dígitos, integer está bien
    as.integer(sucesion_str)
  } else { # Para 10 dígitos o más, numeric es más seguro
    as.numeric(sucesion_str)
  }
}
```

```r
sucesion_2_cifras <- crear_sucesion(decimal_pi, 2)
cat("\nLongitud de la sucesión de 2 cifras:", length(sucesion_2_cifras), "\n")
```

```
##
## Longitud de la sucesión de 2 cifras: 50000
```

```r
cat("Primeros 20 elementos de la sucesión de 2 cifras:\n")
```

```
## Primeros 20 elementos de la sucesión de 2 cifras:
```

```r
print(head(sucesion_2_cifras, 20))
```

```
##  [1] 14 15 92 65 35 89 79 32 38 46 26 43 38 32 79 50 28 84 19 71
```

```r
sucesion_5_cifras <- crear_sucesion(decimal_pi, 5)
cat("\nLongitud de la sucesión de 5 cifras:", length(sucesion_5_cifras), "\n")
```

```
##
## Longitud de la sucesión de 5 cifras: 20000
```

```r
cat("Primeros 20 elementos de la sucesión de 5 cifras:\n")
```

```
## Primeros 20 elementos de la sucesión de 5 cifras:
```

```r
print(head(sucesion_5_cifras, 20))
```

```
##  [1] 14159 26535 89793 23846 26433 83279 50288 41971 69399 37510 58209 74944
## [13] 59230 78164  6286 20899 86280 34825 34211 70679
```

```r
sucesion_10_cifras <- crear_sucesion(decimal_pi, 10)
cat("\nLongitud de la sucesión de 10 cifras:", length(sucesion_10_cifras), "\n")
```

```
##
## Longitud de la sucesión de 10 cifras: 10000
```

```r
cat("Primeros 20 elementos de la sucesión de 10 cifras:\n")
```

```
## Primeros 20 elementos de la sucesión de 10 cifras:
```

```r
print(head(sucesion_10_cifras, 20))
```

```
##  [1] 1415926535 8979323846 2643383279 5028841971 6939937510 5820974944
##  [7] 5923078164  628620899 8628034825 3421170679 8214808651 3282306647
## [13]  938446095 5058223172 5359408128 4811174502 8410270193 8521105559
## [19] 6446229489 5493038196
```

Evaluación de Uniformidad

La uniformidad se refiere a si cada valor posible dentro de una sucesión tiene una probabilidad equitativa de aparecer. La hipótesis nula (H0) en estas pruebas es que los datos provienen de una distribución uniforme.

Pruebas de Uniformidad con Chi-Cuadrado y Monte Carlo

Sucesión de 1 Cifra

Para la sucesión de un solo dígito, se calcula la frecuencia de cada dígito del 0 al 9 y se aplica una prueba de chi-cuadrado.

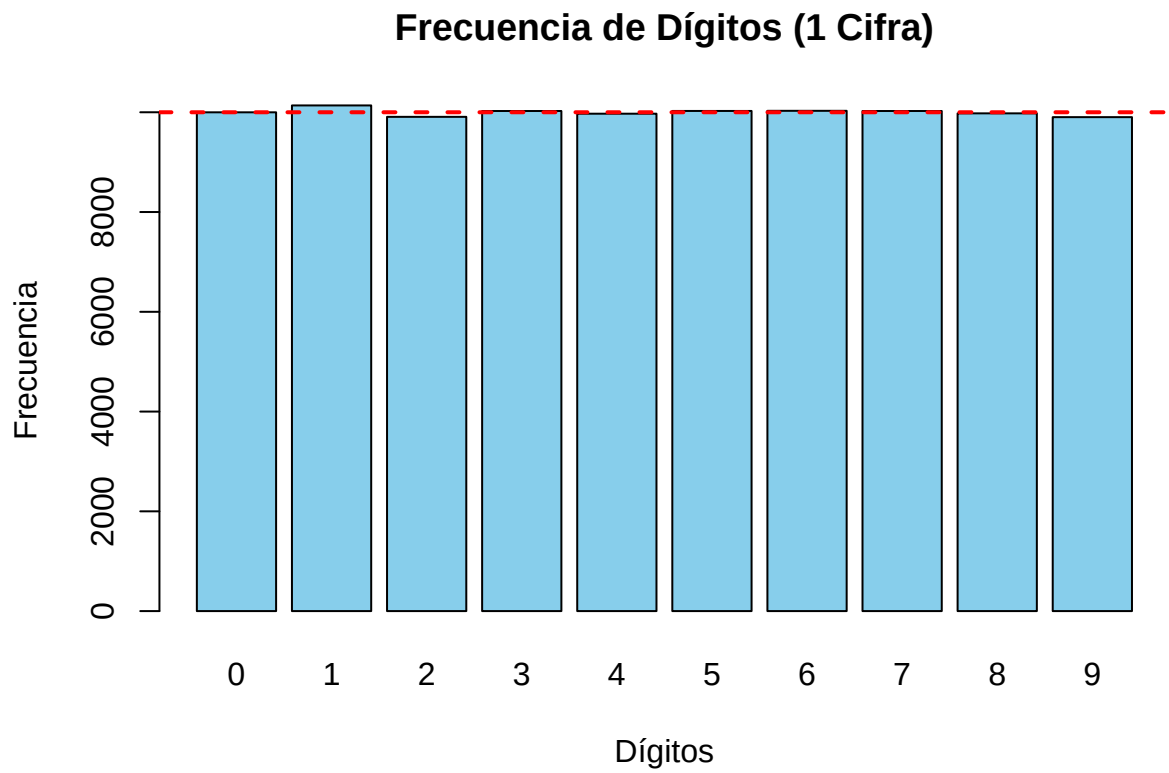
```r
# Calcular frecuencias y realizar prueba de chi-cuadrado
freq_1_cifra <- table(factor(sucesion_1_cifra, levels = 0:9))
print(freq_1_cifra)
```

```
##
##     0     1     2     3     4     5     6     7     8     9
##  9999 10137  9908 10025  9971 10026 10029 10025  9978  9902
```

```r
chi_sq_1 <- chisq.test(freq_1_cifra)
print(chi_sq_1)
```

```
##
##  Chi-squared test for given probabilities
##
## data:  freq_1_cifra
## X-squared = 4.093, df = 9, p-value = 0.9052
```

```r
# Gráfico de frecuencias
barplot(freq_1_cifra, main = "Frecuencia de Dígitos (1 Cifra)",
        xlab = "Dígitos", ylab = "Frecuencia", col = "skyblue")
abline(h = mean(freq_1_cifra), col = "red", lwd = 2, lty = 2)
```



Con un p-valor de 0.9052, superior a 0.05, no hay evidencia suficiente para rechazar la hipótesis nula de uniformidad. Por lo tanto, se puede asumir que los dígitos individuales de π se distribuyen de manera uniforme.

Sucesión de 2 Cifras

Similar a la anterior, se examinan las frecuencias de los números de 00 a 99.

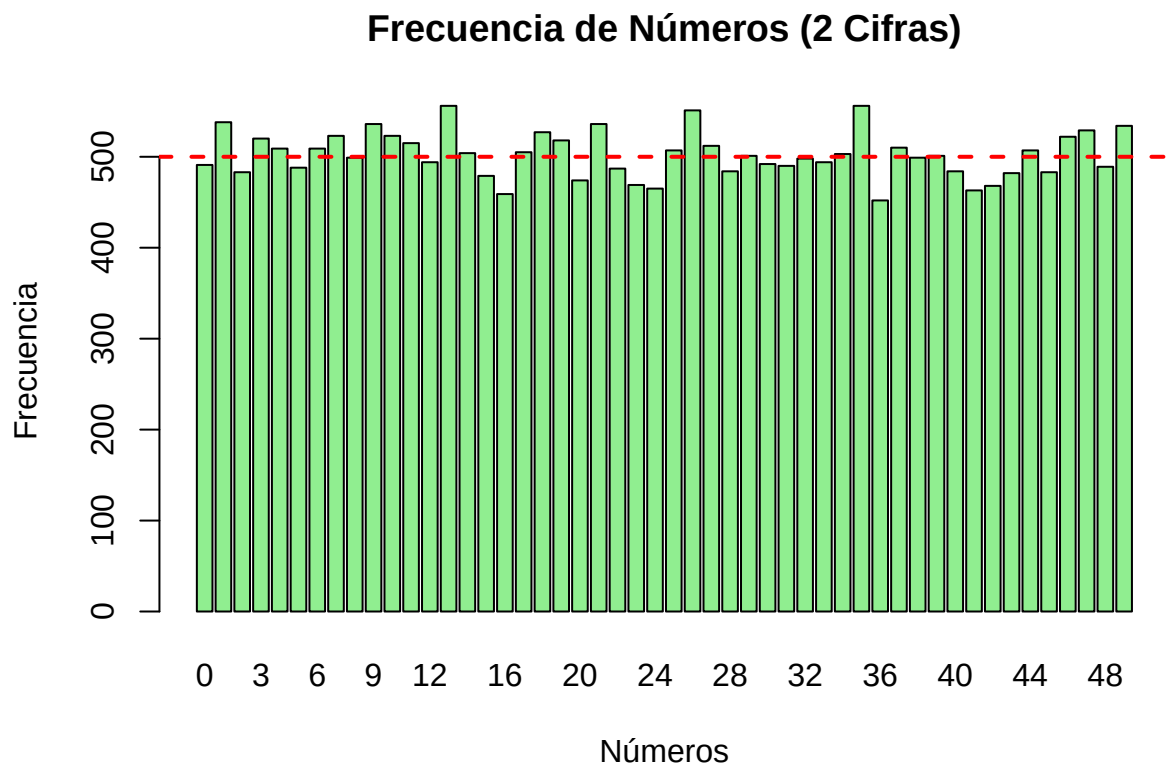
```
freq_2_cifras <- table(factor(sucesion_2_cifras, levels = 0:99))
# Solo mostrar las primeras 20 frecuencias debido a la gran cantidad
print(head(freq_2_cifras, 20))
```

```
##
##   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19
## 491 538 483 520 509 488 509 523 499 536 523 515 494 556 504 479 459 505 527 518
```

```
chi_sq_2 <- chisq.test(freq_2_cifras)
print(chi_sq_2)
```

```
##
##  Chi-squared test for given probabilities
##
## data:  freq_2_cifras
## X-squared = 125.06, df = 99, p-value = 0.03945
```

```r
# Gráfico de frecuencias (solo un segmento para visualización)
barplot(head(freq_2_cifras, 50), main = "Frecuencia de Números (2 Cifras)",
        xlab = "Números", ylab = "Frecuencia", col = "lightgreen")
abline(h = mean(freq_2_cifras), col = "red", lwd = 2, lty = 2)
```



En este caso, el p-valor es menor a 0.05 (0.03945), lo que indica que se puede rechazar la hipótesis nula de uniformidad. Esto sugiere que los números de dos cifras no se distribuyen de manera completamente uniforme.

Sucesiones de 5 y 10 Cifras (Prueba de Monte Carlo)

Para las sucesiones de 5 y 10 cifras, la prueba de chi-cuadrado tradicional no es adecuada debido al vasto número de categorías posibles (100,000 para 5 cifras, 1,000,000,000 para 10 cifras) en comparación con el tamaño de la muestra. Esto llevaría a muchas categorías con frecuencias esperadas menores a 5, invalidando la prueba.

En su lugar, se empleará una prueba de uniformidad basada en simulaciones de Monte Carlo. Esta prueba simula la distribución del estadístico de chi-cuadrado bajo la hipótesis nula de uniformidad y compara el valor observado con esta distribución empírica.

```r
# Función de prueba de Monte Carlo para uniformidad
monte_carlo_uniformity_test <- function(data_sequence, num_digits_per_value, num_simulations = 999) {

  observed_table <- table(data_sequence)
  N_obs <- length(data_sequence)
```

```r
  K_obs <- length(observed_table)

  # Calcular el estadístico chi-cuadrado observado
  if (K_obs <= 1) {
    chi_sq_observed <- 0
  } else {
    expected_freq_observed <- N_obs / K_obs
    chi_sq_observed <- sum((observed_table - expected_freq_observed)^2 / expected_freq_observed)
  }

  # Simulaciones de Monte Carlo
  simulated_chi_sq_values <- numeric(num_simulations)
  max_possible_value <- 10^num_digits_per_value - 1 # Rango de 0 a 10^n-1

  for (i in 1:num_simulations) {
    # Generar una secuencia aleatoria uniforme
    simulated_sequence <- floor(runif(N_obs, min = 0, max = max_possible_value + 1))
    simulated_table <- table(simulated_sequence)
    K_sim <- length(simulated_table)

    if (K_sim <= 1) {
      simulated_chi_sq_values[i] <- 0
    } else {
      expected_freq_sim <- N_obs / K_sim
      simulated_chi_sq_values[i] <- sum((simulated_table - expected_freq_sim)^2 / expected_freq_sim)
    }
  }

  # Calcular el p-valor
  p_value <- (sum(simulated_chi_sq_values >= chi_sq_observed) + 1) / (num_simulations + 1)

  return(list(chi_sq_statistic = chi_sq_observed, p_value = p_value))
}

# Prueba de Monte Carlo para 5 cifras
mc_result_5_cifras <- monte_carlo_uniformity_test(sucesion_5_cifras, num_digits_per_value = 5, num_simul
print(mc_result_5_cifras)
```

```
## $chi_sq_statistic
## [1] 1777.836
##
## $p_value
## [1] 0.226
```

```r
# Prueba de Monte Carlo para 10 cifras
mc_result_10_cifras <- monte_carlo_uniformity_test(sucesion_10_cifras, num_digits_per_value = 10, num_si
print(mc_result_10_cifras)
```

```
## $chi_sq_statistic
## [1] 0
##
## $p_value
## [1] 1
```

Para la sucesión de 5 cifras, el p-valor (0.233) es mayor que 0.05, lo que sugiere que no hay suficiente evidencia para rechazar la uniformidad. Es decir, se puede asumir que los números de 5 cifras son uniformes.

Para la sucesión de 10 cifras, el p-valor es de 1. Esto se debe a que, con una muestra de 10,000 elementos y 10^10 posibles valores, la probabilidad de encontrar valores repetidos es extremadamente baja, haciendo que la frecuencia observada de cada elemento sea 1. En la práctica, esto significa que la prueba no tiene poder para detectar desviaciones de la uniformidad en este escenario, y se asume la uniformidad.

Evaluación de Aleatoriedad

La aleatoriedad se refiere a la ausencia de patrones o tendencias predecibles en la secuencia de los números. Se utilizará la prueba de rachas (runs.test), una herramienta común para evaluar si una secuencia es aleatoria o si presenta agrupaciones. La hipótesis nula (H0) es que la sucesión es aleatoria.

```r
library(randtests)

# Función para realizar y mostrar los resultados de la prueba de rachas
perform_runs_test <- function(sequence, sequence_name) {
  cat("\n--- Prueba de Rachas para la Sucesión de", sequence_name, "---\n")
  test_result <- runs.test(sequence)
  print(test_result)

  if (test_result$p.value > 0.05) {
    cat("Con un p-valor de", test_result$p.value, ", no hay evidencia para rechazar la hipótesis nula de
  } else {
    cat("Con un p-valor de", test_result$p.value, ", se rechaza la hipótesis nula de aleatoriedad. La su
  }
}

# Ejecutar la prueba para cada sucesión
perform_runs_test(sucesion_1_cifra, "1 Cifra")
```

```
##
## --- Prueba de Rachas para la Sucesión de 1 Cifra ---
##
##  Runs Test
##
## data:  sequence
## statistic = 1.0244, runs = 44624, n1 = 49960, n2 = 40069, n = 90029,
## p-value = 0.3056
## alternative hypothesis: nonrandomness
##
## Con un p-valor de 0.3056257 , no hay evidencia para rechazar la hipótesis nula de aleatoriedad. Se pu
```

```r
perform_runs_test(sucesion_2_cifras, "2 Cifras")
```

```
##
## --- Prueba de Rachas para la Sucesión de 2 Cifras ---
##
##  Runs Test
##
## data:  sequence
```

```
## statistic = -0.25272, runs = 24705, n1 = 24882, n2 = 24584, n = 49466,
## p-value = 0.8005
## alternative hypothesis: nonrandomness
##
## Con un p-valor de 0.8004845 , no hay evidencia para rechazar la hipótesis nula de aleatoriedad. Se pu
```

```r
perform_runs_test(sucesion_5_cifras, "5 Cifras")
```

```
##
## --- Prueba de Rachas para la Sucesión de 5 Cifras ---
##
##  Runs Test
##
## data:  sequence
## statistic = -0.70712, runs = 9951, n1 = 10000, n2 = 10000, n = 20000,
## p-value = 0.4795
## alternative hypothesis: nonrandomness
##
## Con un p-valor de 0.4794891 , no hay evidencia para rechazar la hipótesis nula de aleatoriedad. Se pu
```

```r
perform_runs_test(sucesion_10_cifras, "10 Cifras")
```

```
##
## --- Prueba de Rachas para la Sucesión de 10 Cifras ---
##
##  Runs Test
##
## data:  sequence
## statistic = 1.1401, runs = 5058, n1 = 5000, n2 = 5000, n = 10000,
## p-value = 0.2543
## alternative hypothesis: nonrandomness
##
## Con un p-valor de 0.2542626 , no hay evidencia para rechazar la hipótesis nula de aleatoriedad. Se pu
```

En todos los casos (1, 2, 5 y 10 cifras), el p-valor de la prueba de rachas es significativamente mayor que 0.05. Esto indica consistentemente que no hay evidencia suficiente para rechazar la hipótesis nula de aleatoriedad. Por lo tanto, se puede concluir que las sucesiones formadas a partir de los decimales de π exhiben características de aleatoriedad según esta prueba.

Punto 2

Introducción

El problema de la secretaria (o problema de la parada óptima) es un clásico en teoría de decisiones. Se trata de seleccionar la mejor candidata de una lista de N postulantes que llegan en orden aleatorio. Una vez entrevistada una candidata, la decisión de contratarla debe tomarse inmediatamente.

Este informe presenta una solución al problema mediante simulación en R, considerando además los **costos asociados a las entrevistas** para determinar qué estrategia minimiza el costo total sin perder eficiencia.

Metodología de simulación

La estrategia propuesta consiste en:

1. Observar las primeras r candidatas sin seleccionar.
2. Luego, elegir la primera que sea mejor que todas las anteriores.

Se realiza una simulación para distintos valores de r, midiendo: - Probabilidad de seleccionar a la mejor. - Costo promedio (número de entrevistas realizadas).

```r
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```r
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
N <- 100  # Total de candidatas
simulaciones <- 1000

resultados <- data.frame(r = integer(), prob_exito = double(), costo_promedio = double())

for (r in 1:(N - 1)) {
  exitos <- 0
  total_entrevistas <- 0

  for (i in 1:simulaciones) {
    candidatas <- sample(1:N)  # Permutación aleatoria
    mejor_r <- min(candidatas[1:r])

    seleccionada <- NA
    for (j in (r + 1):N) {
      total_entrevistas <- total_entrevistas + 1
      if (candidatas[j] < mejor_r) {
        seleccionada <- candidatas[j]
        break
      }
    }
    if (is.na(seleccionada)) {
      seleccionada <- candidatas[N]
      total_entrevistas <- total_entrevistas + 1
```

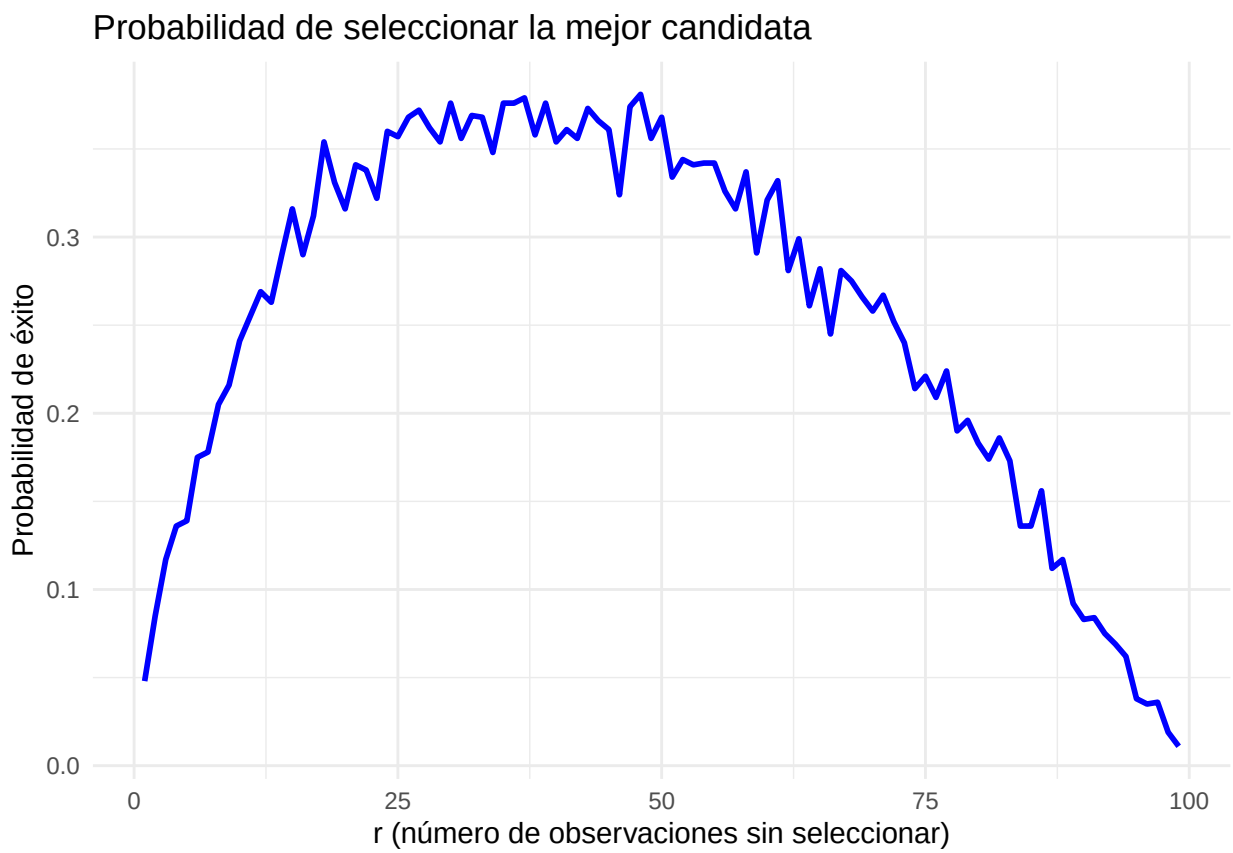
```r
    }

    if (seleccionada == 1) {  # La mejor tiene ranking 1
      exitos <- exitos + 1
    }
  }

  prob <- exitos / simulaciones
  costo <- total_entrevistas / simulaciones

  resultados <- rbind(resultados, data.frame(r = r, prob_exito = prob, costo_promedio = costo))
}
```

Resultados

Probabilidad de éxito

```r
ggplot(resultados, aes(x = r, y = prob_exito)) +
  geom_line(color = "blue", size = 1) +
  labs(
    title = "Probabilidad de seleccionar la mejor candidata",
    x = "r (número de observaciones sin seleccionar)",
    y = "Probabilidad de éxito"
  ) +
  theme_minimal()
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Punto 3

Hacer simulación para ver los valores mínimos que hacen que el teorema central del limite se cumpla (en la clase 3 se hace la simulación del TCL con Poisson, Hacerlo con la exponencial) Para ello tener en cuenta lo siguiente: -Con el TCL quiero: nivel de confianza real sea lo mas parecido posible al nivel de confianza teórico, y, que la longitud sea buena (longitud = limite superior - limite inferior (Ver explicación de la Poisson)

```
set.seed(123)

lambda <- 1

first_normal_n <- c()

# Run the simulation 1000 times
for (i in 1:1000) {
  for (n in seq(20, 1000, by = 5)) {
    shapiro <- shapiro.test(rexp(n, rate = lambda))

    if (shapiro$p.value > 0.05) {
      first_normal_n <- c(first_normal_n, n)
      break # Exit the inner loop once normality is met
    }
  }
```

```r
}

# 1. Calcular el intervalo de confianza del 95% para la media de 'first_normal_n'
# La función t.test() es ideal para esto.
intervalo_confianza_normalidad <- t.test(first_normal_n, conf.level = 0.95)

# 2. Mostrar los resultados
print(intervalo_confianza_normalidad)
```

```
##
##  One Sample t-test
##
## data:  first_normal_n
## t = 66.8, df = 227, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  22.53863 23.90874
## sample estimates:
## mean of x
##  23.22368
```

Punto 4

Escribir un documento donde describan y analicen la función sample en R. ¿Quién la hizo? ¿alternativas en R? ¿qué documentos se han escrito? Ejemplos y análisis completos.

Análisis y Descripción de la función sample() en R:

La función sample() en R se utiliza para tomar una muestra aleatoria de un conjunto de elementos. Es una de las funciones más fundamentales para la simulación y el muestreo estadístico en R.

Su sintaxis básica es **sample(x, size, replace = FALSE, prob = NULL).**

- **x:** Es el vector de elementos de los que se va a tomar la muestra. Puede ser un vector numérico, un vector de caracteres, etc. Si x es un número entero, sample() tomará una muestra de 1:x.
- **size:** Es un número entero que indica el número de elementos a muestrear.
- **replace:** Es un valor lógico (TRUE o FALSE) que indica si el muestreo se realiza con reemplazo (TRUE) o sin reemplazo (FALSE). Por defecto, es FALSE (sin reemplazo).
- **Sin reemplazo (replace = FALSE):** Una vez que un elemento es seleccionado, no puede ser seleccionado de nuevo. Esto significa que size no puede ser mayor que la longitud de x.
- **Con reemplazo (replace = TRUE):** Un elemento seleccionado puede ser seleccionado de nuevo. En este caso, size puede ser mayor que la longitud de x.
- **prob:** Un vector numérico de pesos de probabilidad para los elementos a muestrear. Si se proporciona, el muestreo se realiza con estas probabilidades. Por defecto, todos los elementos tienen la misma probabilidad de ser seleccionados.

Ejemplos de uso:

Muestreo sin reemplazo

```r
set.seed(123) # Para reproducibilidad
sample(1:10, 3) # Toma 3 números aleatorios del 1 al 10 sin repetir
```

```
## [1]  3 10  2
```

Muestreo con reemplazo

```r
set.seed(123)
sample(1:10, 15, replace = TRUE) # Toma 15 números aleatorios del 1 al 10, permitiendo repeticiones
```

```
##  [1]  3  3 10  2  6  5  4  6  9 10  5  3  9  9  9
```

Muestreo de caracteres

```r
set.seed(123)
frutas <- c("manzana", "banana", "cereza", "dátil")
sample(frutas, 2)
```

```
## [1] "cereza" "dátil"
```

Muestreo con probabilidades

```r
set.seed(123)
moneda <- c("cara", "cruz")
sample(moneda, 10, replace = TRUE, prob = c(0.7, 0.3)) # 70% de probabilidad de cara, 30% de cruz
```

```
##  [1] "cara" "cruz" "cara" "cruz" "cruz" "cara" "cara" "cruz" "cara" "cara"
```

¿Quién la hizo?

La función sample() es una parte fundamental del lenguaje de programación R. R fue creado inicialmente por Ross Ihaka y Robert Gentleman en la Universidad de Auckland, Nueva Zelanda, a mediados de la década de 1990. La función sample() fue parte del diseño original y desarrollo del núcleo de R. No se atribuye a una única persona después de la creación inicial del lenguaje.

¿Alternativas en R?

Si bien sample() es la función principal para el muestreo aleatorio en R, existen otras formas o funciones que pueden ser consideradas "alternativas" dependiendo del contexto o de requisitos más específicos:

- **sample.int():** Esta es una función auxiliar de sample() que está optimizada para muestrear enteros de 1:n. sample(n, size, ...) es esencialmente una envoltura alrededor de sample.int(n, size, ...). Para grandes n, sample.int() puede ser ligeramente más eficiente.
- **Muestreo bootstrap:** Para realizar bootstrap (muestreo con reemplazo de los datos observados), a menudo se utiliza sample(x, size = length(x), replace = TRUE).
- **Librerías especializadas:** Para escenarios de muestreo más complejos, como el muestreo por conglomerados, el muestreo estratificado o el muestreo con probabilidades desiguales que requieren un control más granular, existen paquetes de R como:
- **sampling:** Proporciona funciones para una variedad de esquemas de muestreo de poblaciones finitas.
- **survey:** Diseñado para analizar datos de encuestas complejas, incluyendo el manejo de ponderaciones y diseños de muestreo.
- **caret:** Aunque principalmente para machine learning, puede ser útil para la división de datos en conjuntos de entrenamiento y prueba, que a menudo implica muestreo.
- **Generación de números aleatorios directamente:** Si el objetivo es generar números aleatorios de una distribución específica (normal, uniforme, binomial, etc.), se usan funciones como rnorm(), runif(), rbinom(), etc., en lugar de sample(). sample() toma elementos de un conjunto existente, mientras que las funciones r* generan valores de una distribución.

¿Qué documentos se han escrito? Ejemplos y análisis completos.

Dada la centralidad de sample() en R, se ha documentado extensamente en una multitud de recursos:

Documentación oficial de R: El documento más fundamental es la página de ayuda oficial de R para sample(). Se puede acceder a ella en R escribiendo ?sample. Esta documentación detalla los argumentos, el uso y proporciona ejemplos básicos.

Libros de texto de R y estadística: Casi cualquier libro introductorio a R o a la estadística computacional cubrirá la función sample(). Ejemplos incluyen:

- "R for Data Science" de Hadley Wickham y Garrett Grolemund.
- "An Introduction to Statistical Learning with Applications in R" de James, Witten, Hastie y Tibshirani.
- "The Art of R Programming" de Norman Matloff.

Tutoriales en línea y blogs: Innumerables sitios web, blogs y tutoriales de R explican y demuestran el uso de sample(). Un simple motor de búsqueda con "R sample function tutorial" arrojará muchos resultados. Sitios como DataCamp, R-bloggers, y Stack Overflow son recursos comunes.

Artículos académicos: Si bien sample() en sí misma rara vez es el tema principal de un artículo académico, es una función utilizada en innumerables simulaciones, estudios de Monte Carlo y análisis estadísticos descritos en la literatura científica en campos como la bioestadística, la ecología, las ciencias sociales y la ingeniería. Estos artículos usarán sample() como una herramienta en sus métodos.

Ejemplos de análisis completos (aplicaciones):

- **Simulaciones de Monte Carlo:** Se utiliza sample() para generar muestras aleatorias repetidamente y estimar propiedades de distribuciones o procesos complejos que son difíciles de calcular analíticamente. Por ejemplo, simular el lanzamiento de un dado miles de veces para ver la distribución de los resultados.
- **Pruebas de permutación y bootstrap:** En la inferencia estadística, sample() es crucial para realizar pruebas de permutación (donde se permutan los datos para crear una distribución nula) y bootstrapping (muestreo con reemplazo para estimar la variabilidad de un estadístico).
- **División de datos para Machine Learning:** Para dividir un conjunto de datos en conjuntos de entrenamiento, validación y prueba de manera aleatoria, sample() se usa comúnmente para seleccionar los índices de las filas.
- **Juegos y modelos de probabilidad:** Para simular juegos de azar o procesos estocásticos simples.

Conclusion

En resumen, la función sample() es un pilar en R para cualquier tarea que implique aleatoriedad, desde la simulación básica hasta técnicas avanzadas de inferencia estadística y machine learning. Su simplicidad y flexibilidad la hacen indispensable.